

CLOUD NATIVE SUMMIT • MUNICH 2026

Why LLMs Become Expensive in Production

How Control-Plane Architecture Fixes It

Hiranya Abeyratne | Technical Lead @ WSO2

June 29–30, 2026 • Munich, Germany



A Fraud Alert Arrives

Fraud Alert Card

ALERT #FRD-88421

HIGH RISK

- New bank account detected
- Transfers across 3 countries
- Shared device fingerprint accounts
- High-risk crypto destination
- Linked beneficiaries identified

95/100 SEVERE RISK

2024-05-19 17:35:37 UTC

AI Fraud Investigation Agent

- Fraud Alert Triage
- Fraud Alert Investigation
- Linked Account Analysis
- Temporary Account Restriction
- Suspicious Activity Report Draft

A Fraud Alert Arrives

Fraud Alert Card

ALERT #FRD-88421

HIGH RISK

- New bank account detected
- Transfers across 3 countries
- Shared device fingerprint accounts
- High-risk crypto destination
- Linked beneficiaries identified

95/100 SEVERE RISK

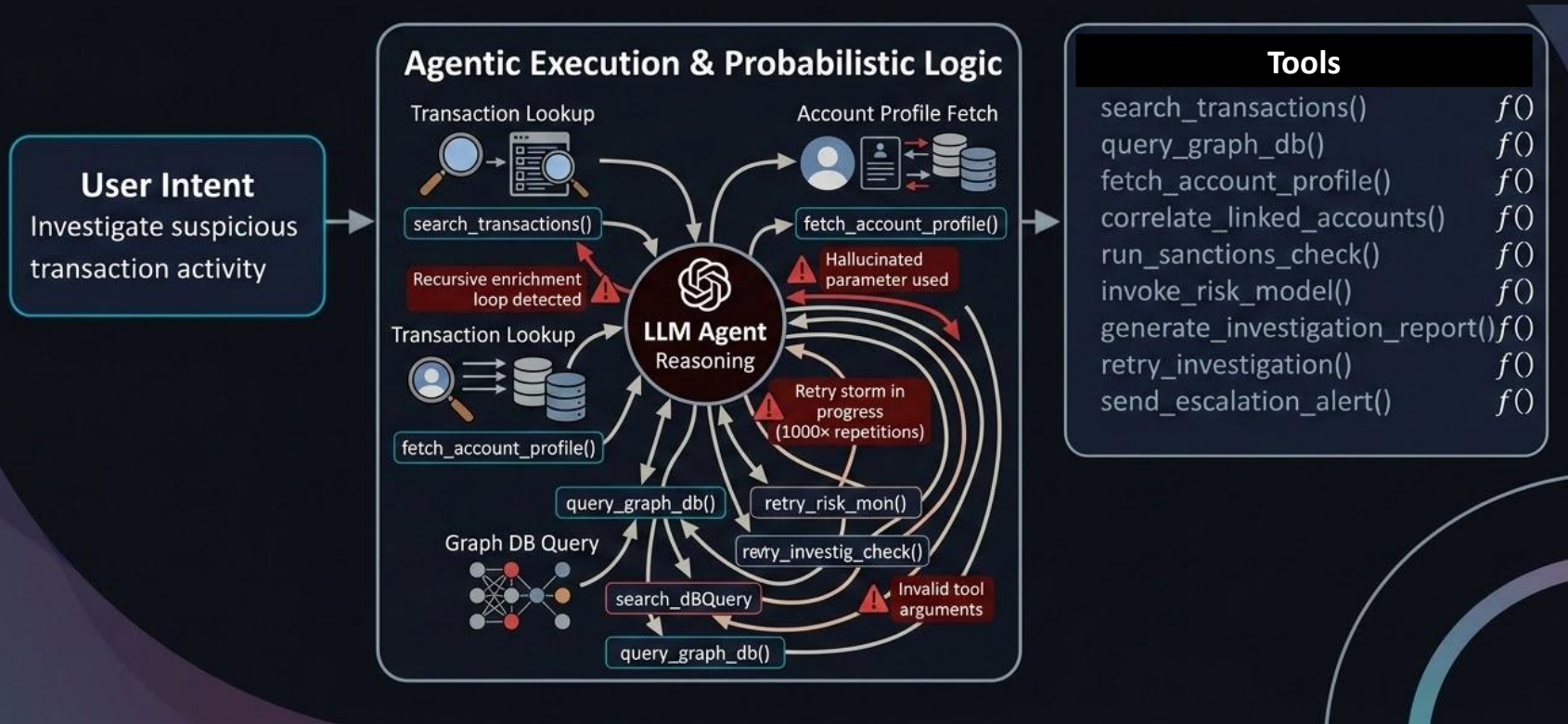
2024-05-19 17:35:37 UTC

AI Fraud Investigation Agent

- Fraud Alert Triage**
- Fraud Alert Investigation**
- Linked Account Analysis**
- Temporary Account Restriction**
- Suspicious Activity Report Draft**

Behind the Scenes: What the Agent Is Actually Doing

The LLM receives low-level execution primitives as tools — and must probabilistically orchestrate the workflow itself.



The Token Economics of One Fraud Alert Investigation

Token economics for an LLM-orchestrated fraud investigation workflow

· Priced at Claude Opus 4.8 standard rates (\$5 in / \$25 out per MTok). Costs vary by model.

Step	What the LLM is doing	Token usage	Estimated Cost
1	Load Fraud Case Data Reads fraud alert + tool manifests → decides to call graph DB API	~30K input + ~2K output	\$0.14
2	Correlate Linked Accounts Reads previous API responses + tool manifests → decides correlation calls → generates call parameters	~58K input + ~5K output	\$0.29
3	Sanctions & Risk Scoring Reads accumulated investigation context + tool manifests → invokes sanctions/risk scoring services → reasons over responses	~86K input + ~4K output	\$0.37
4	Generate Investigation Report Reads accumulated execution history + investigation evidence → generates investigator report	~100K input + ~14K output	\$0.57

The Token Economics of One Fraud Alert Investigation

Token economics for an LLM-orchestrated fraud investigation workflow

· Priced at Claude Opus 4.8 standard rates (\$5 in / \$25 out per MTok). Costs vary by model.

Step	What the LLM is doing	Token usage	Estimated Cost
1	Load Fraud Case Data Reads fraud alert + tool manifests → decides to call graph DB API	~30K input + ~2K output	\$0.14
2	Correlate Linked Accounts Reads previous API responses + tool manifests → decides correlation calls → generates call parameters	~58K input + ~5K output	\$0.29
3	Sanctions & Risk Scoring Reads accumulated investigation context + tool manifests → invokes sanctions/risk scoring services → reasons over responses	~86K input + ~4K output	\$0.37
4	Generate Investigation Report Reads accumulated execution history + investigation evidence → generates investigator report	~100K input + ~14K output	\$0.57
TOTAL · ~274K input · ~25K output			\$2.00 / investigation

The \$2.00M Weekend

What happens when an agentic retry loop goes undetected



Root cause: A recursive orchestration loop repeatedly re-executed the same investigation flow. No loop detection. No spend cap. No kill switch.

Autonomous orchestration

amplifies runaway infrastructure costs at a speed no human operator can catch manually.

What Actually Drives AI Execution Cost

Not the model. Not the prompt. But the execution architecture

01

Context Accumulation

Every workflow step expands future token cost

02

Autonomous Retries

Recursive retries amplify inference and API spend

03

Workflow Depth

More orchestration steps mean more reasoning cycles

04

Tool Fan-Out

Agents calling multiple tools multiply execution cost

05

Model Selection

Wrong model routing inflates runtime cost



What Actually Drives AI Execution Cost

Not the model. Not the prompt. But the execution architecture

01

Context Accumulation

Every workflow step expands future token cost

02

Autonomous Retries

Recursive retries amplify inference and API spend

03

Workflow Depth

More orchestration steps mean more reasoning cycles

04

Tool Fan-Out

Agents calling multiple tools multiply execution cost

05

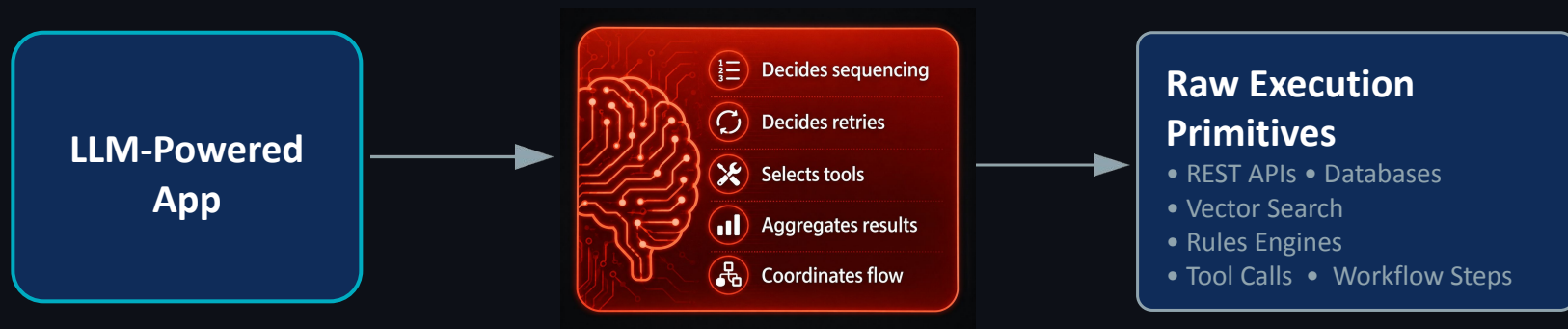
Model Selection

Wrong model routing inflates runtime cost

None of these are model problems. All five are architecture decisions.

The Core Problem: LLM as the Execution Layer

Probabilistic reasoning + autonomous tool selection + direct access to raw primitives create governance risk

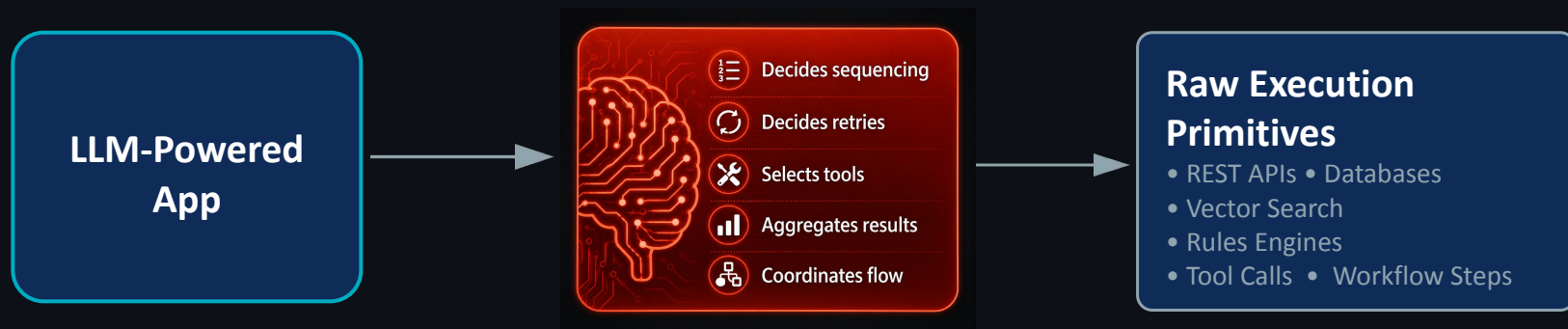


CONSEQUENCES

- ✗ Loss of deterministic control flow
- ✗ Hidden execution paths
- ✗ Recursive execution loops
- ✗ No centralized governance
- ✗ Token explosion via context accumulation

The Core Problem: LLM as the Execution Layer

Probabilistic reasoning + autonomous tool selection + direct access to raw primitives create governance risk



CONSEQUENCES

- ✗ Loss of deterministic control flow
- ✗ Recursive execution loops
- ✗ Token explosion via context accumulation
- ✗ Hidden execution paths
- ✗ No centralized governance

Ungoverned Probabilistic Orchestration

The Impact: Operational Failure Modes

Operational failures emerge when execution decisions are delegated to the model



Identity Gap

Who initiated this? Which session? Which user?
Which runtime identity?



Retry Storms

The system retries on failure, creating 1000× repeated execution.



Invalid Tool Arguments

LLMs invent tool arguments on the fly. Downstream systems receive corrupted inputs.

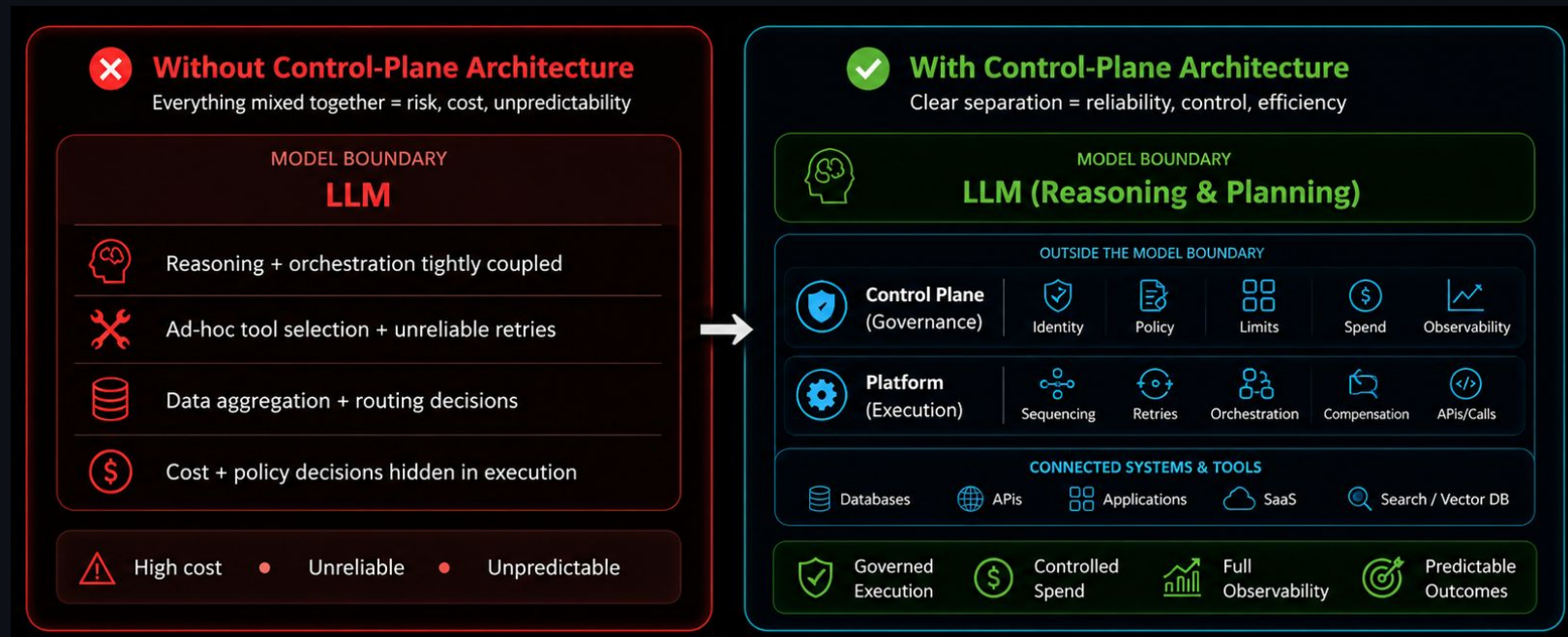


No Spend Ceiling

Without session-level budget enforcement, a single execution can exhaust a monthly budget in hours.

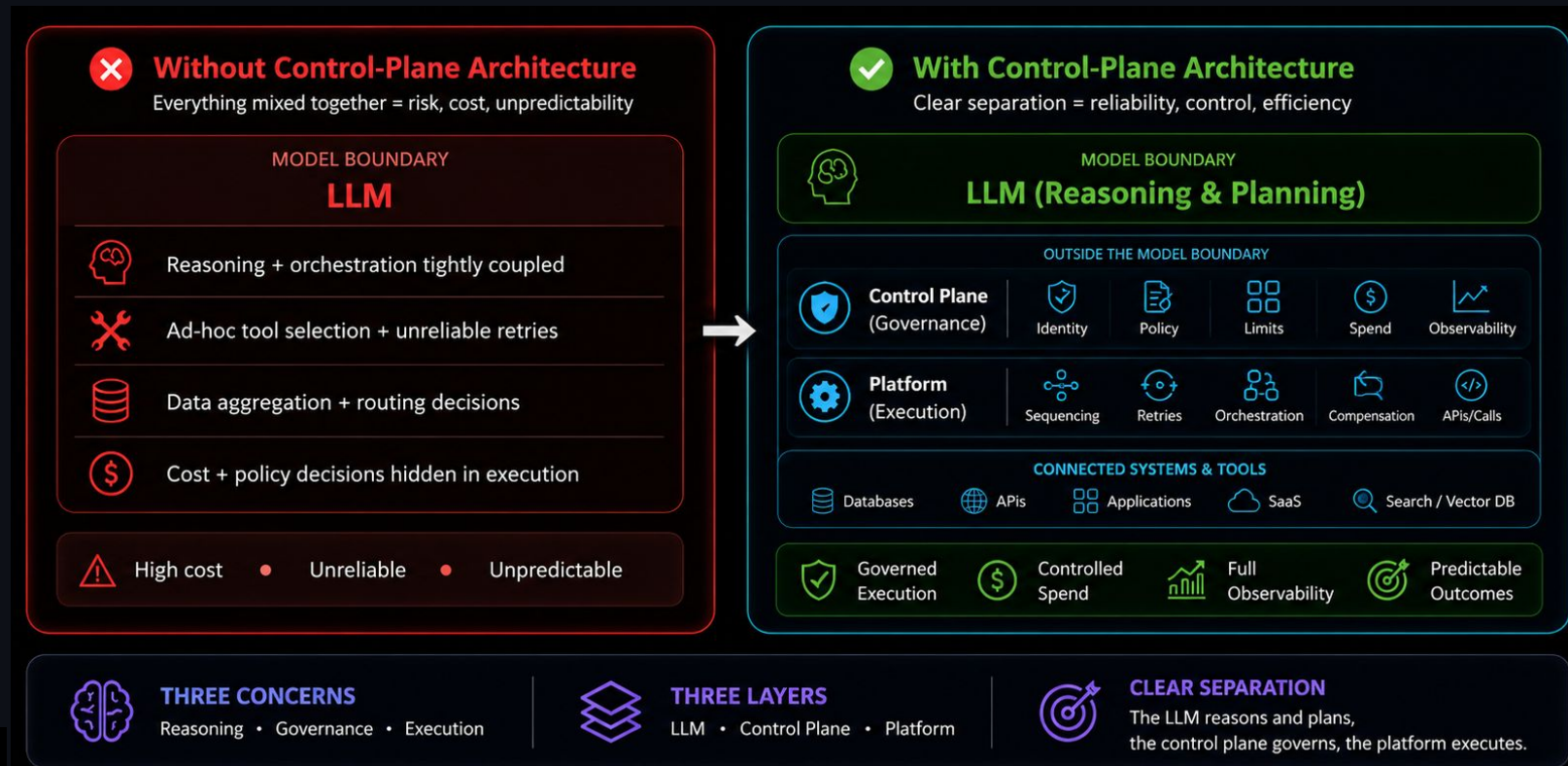
The Fix: Control-Plane Architecture for Governed Execution

Separation of concerns for LLM powered systems. Reasoning, governance, and execution become three independent layers



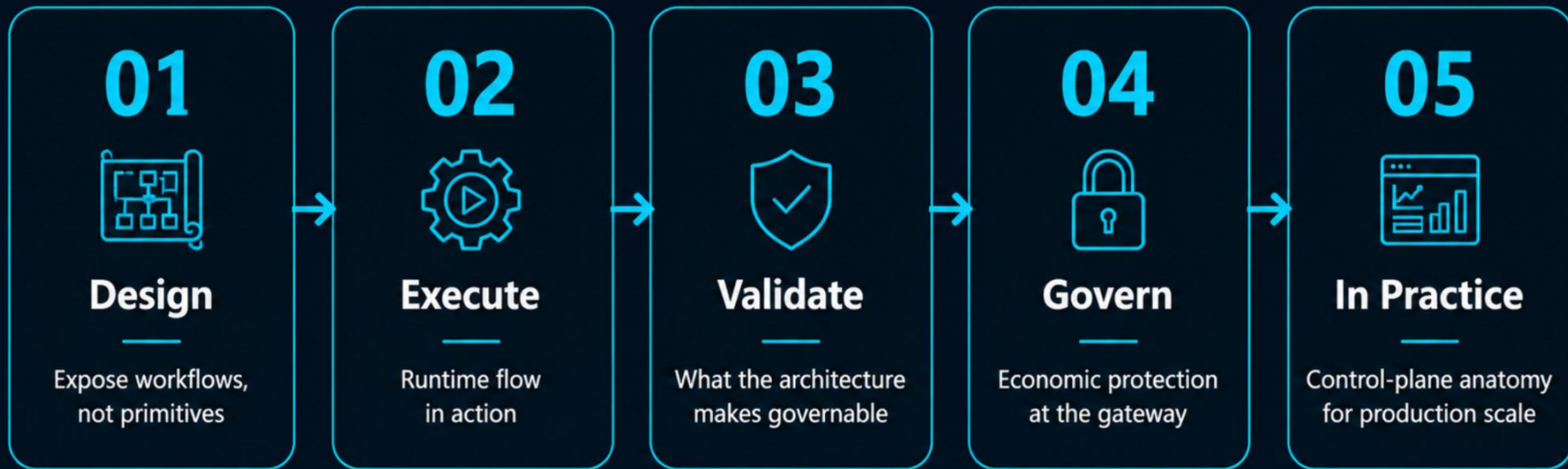
The Fix: Control-Plane Architecture for Governed Execution

Separation of concerns for LLM powered systems. Reasoning, governance, and execution become three independent layers



The Solution Path: From LLM Orchestration to Governed Execution

Five steps from architectural principle to production implementation.



By Step 5, the five cost drivers become bounded, observable, and governable

Step 1. Design: Expose Workflows, Not Primitives

Use Case: A fraud investigation agent needs to support a fraud alert investigation workflow

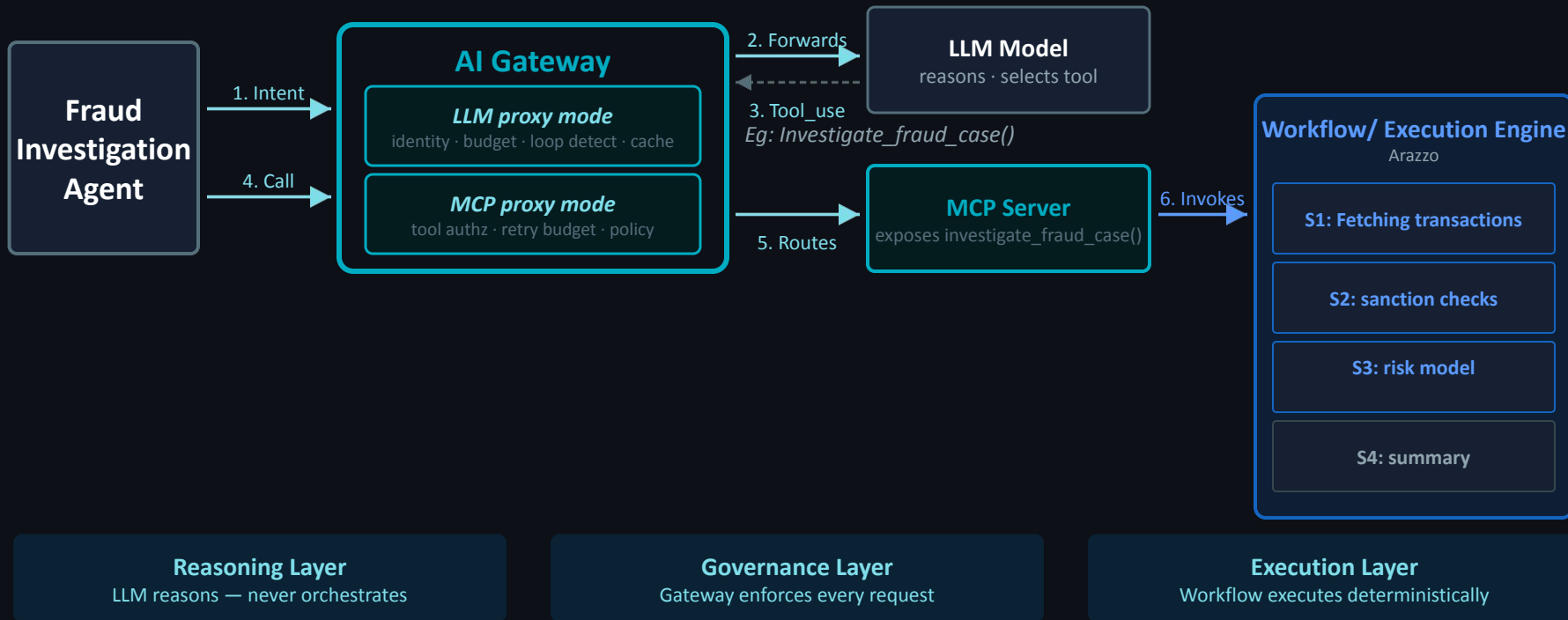
Approach: Expose the workflow as a single MCP tool, with deterministic execution defined in **Arazzo**



Expose governed capabilities. Hide the implementation from the model

Step 2. Execute: Runtime Flow in Action

The LLM interprets the intent and selects the appropriate execution path. The platform executes the workflow deterministically under gateway governance.



One tool call. Gateway-mediated execution. Deterministic workflow steps.

Step 3. Validate: Architecture Makes the Risks Governable

When orchestration moves outside the model boundary, these risks become visible, observable and governable at the platform level

Token explosion

LLM orchestrating 5 raw primitives. Tool manifests + full context on every call

Deterministic workflows + semantic caching

Hidden execution paths

No identity, no audit trail. Who called what and when?

Single workflow entry point with identity propagation and end-to-end tracing

Retry storms

LLM retries unbounded. Same tool called 1,000×

Retries become bounded via retry policies

Recursive loops

Agent re-executes same investigation flow continuously

Repeated execution patterns are now visible outside the model

Step 4A. Govern: Runtime Controls for Production LLM Systems

Session-level enforcement for cost, retries, loops, and operational boundaries



Token Counting & Cost Estimation

- Real-time token accumulation per session
- Pre-flight cost estimation before execution
- MCP tool usage analytics



Session Spend Tracking

- Session-level spend budgets
- Execution velocity monitoring
- Automatic budget cutoff.



Loop Detection & Kill Switch

- Detect recursive workflow execution
- Detect abnormal execution velocity
- Kill switch for runaway sessions

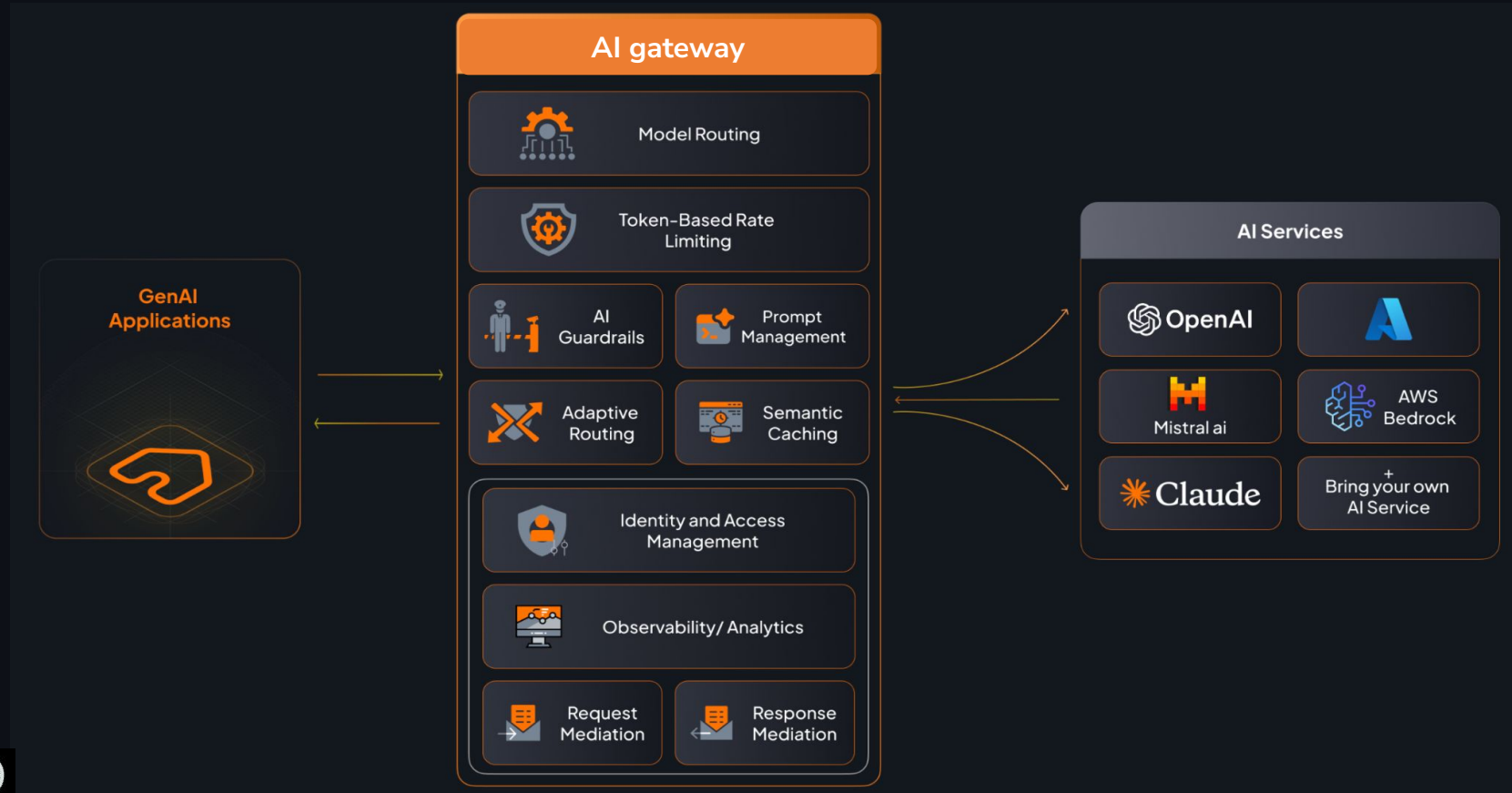


Semantic Workflow Caching & Deduplication

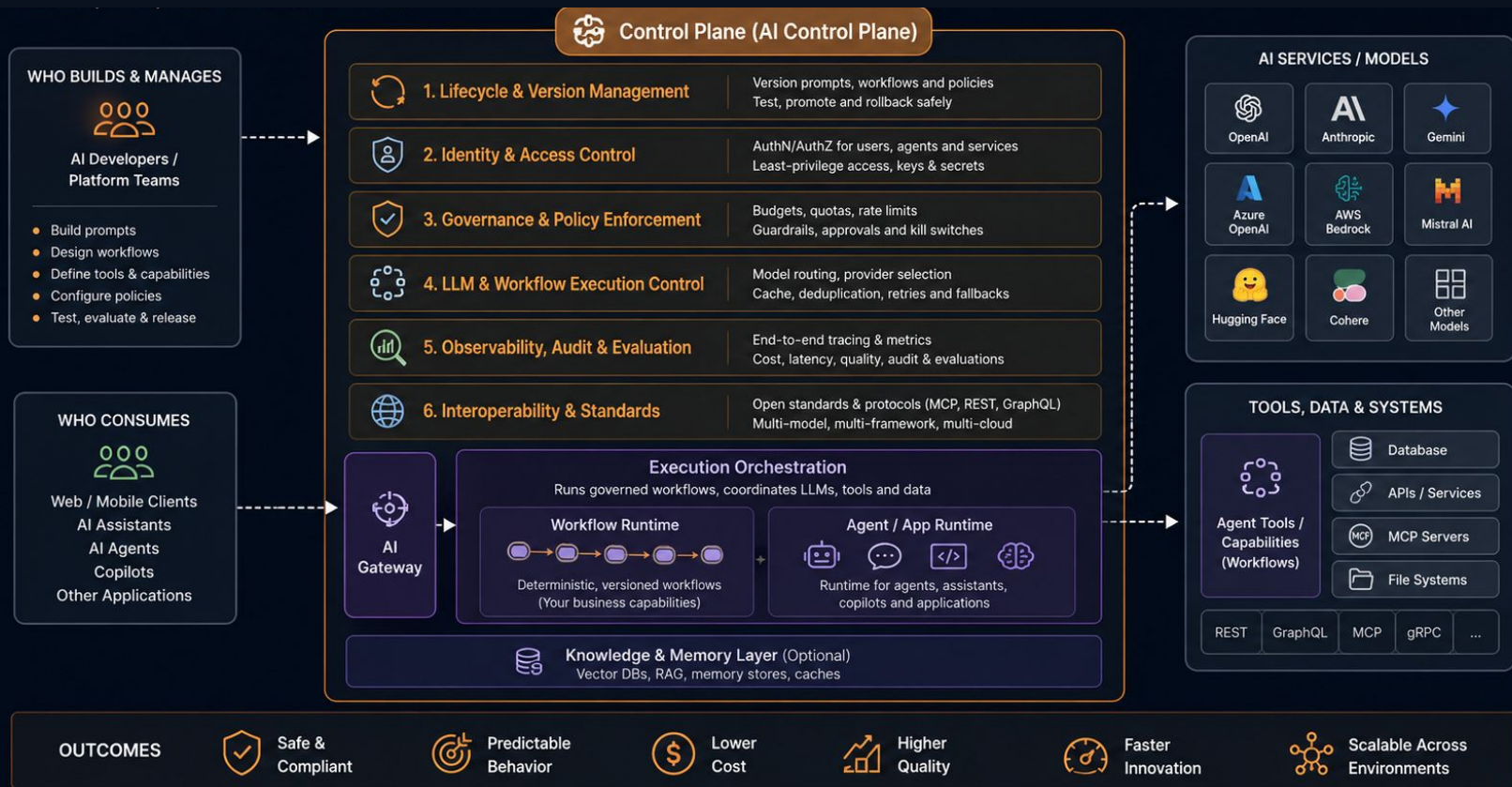
- Cache semantically identical requests
- Deduplicate concurrent tool executions
- Eliminate redundant execution across sessions

Step 4B. Govern: Runtime Governance in Practice

Runtime governance in practice. The AI gateway enforces policies, identity, observability, and execution controls.



Step 5. In Practice: Control-Plane Anatomy for Production Scale



Key Takeaways

01

LLM cost and latency challenges are primarily **architectural**; not model limitations.

02

Treating LLMs as orchestration execution engines causes **token waste**, retry storms, and poor observability.

03

Separate reasoning from orchestration. Keep the LLM focused on intent and reasoning, while **deterministic execution layers run workflows exposed as governed tools**.

04

Control-plane architecture makes agentic execution bounded, observable, and governable by centralizing identity, policy, routing, spend control, and runtime enforcement.

You don't have a model problem. You have an architecture problem



The Model

Reasons



The Control Plane

Governs



The Platform

Executes



Questions?

Resources & Reference Architecture



Session Feedback



Why LLMs Become Expensive in Production – and How
Control-Plane Architecture Fixes It

